

Computational Statistics

Unit 7: Optimisation

Nicolás Rivera

1. Introduction

Optimisation is very important in several fields, including Statistics/Machine Learning

In general, we have a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and we want to find $x^* \in \mathbb{R}^d$ such that $f(x^*)$ is minimum.

Of course we need some restrictions over f such as continuity, etc.

We will cover **some** ideas but Optimisation is a field on its own

The basics

We will mostly consider function that have a couple of derivatives

Partial derivative, gradient and Hessian

Let $U \subseteq \mathbb{R}^d$ be an open set and let $f : U \rightarrow \mathbb{R}$ be a function. The partial derivative of f at points $a \in \mathbb{R}^d$ is given by

$$\frac{\partial}{\partial x_i} f(a) = \lim_{h \rightarrow 0} \frac{f(a + h e_i) - f(a)}{h}$$

The gradient is the vector of partial derivatives, i.e.

$$\nabla f(a) = \left(\frac{\partial}{\partial x_1} f(a), \frac{\partial}{\partial x_2} f(a), \dots, \frac{\partial}{\partial x_d} f(a) \right)$$

The Hessian Matrix H of f at the point a is the $d \times d$ matrix given by

$$H_{ij} = \frac{\partial^2}{\partial x_i \partial x_j} f(a)$$

Sometimes we denote $(Hf)(a)$ but the notation starts getting a bit heavy

A very important property is the following

Theorem

Schwarz's theorem Let $f : U \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$. If f has continuous second order partial derivatives, i.e. if the function

$$a \rightarrow \frac{\partial^2}{\partial x_i \partial x_j} f(a)$$

is continuous at the point a^ , then $(Hf)(a^*)$ is symmetric.*

In general we want to have continuous second order partial derivatives which means that locally, functions are very very smooth.

Example

Let $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ given by

$$f(x, y, z) = z \sin(xy)$$

then

$$\nabla f = (yz \cos(xy), xz \cos(xy), \sin(xy))$$

and

$$(Hf)(x, y, z) = \begin{pmatrix} -y^2 z \sin(xy) & z \cos(xy) - xyz \sin(xy) & y \cos(xy) \\ z \cos(xy) - xyz \sin(xy) & -x^2 z \sin(xy) & x \cos(xy) \\ y \cos(xy) & x \cos(xy) & 0 \end{pmatrix}$$

The basics: critical points

A point $x^* \in \mathbb{R}^d$ is a critical point for f if

$$(\nabla f)(x^*) = 0$$

We say that x^* is a local minimum if exists $\varepsilon > 0$ such that

$$f(x^*) < f(x)$$

for all x such that $\|x^* - x\| < \varepsilon$. The same definition applies for a local maximum.

Theorem

If $f(x^)$ is a local minimum or a local maximum, then x^* is a critical point*

Of course, not all critical points are local minimum or maximum, in which case they are called **saddle points**.

The basics: critical points

A criteria for minimum and maximums is by looking at the Hessian matrix.

Definition (Positive/Negative Definite)

A $d \times d$ matrix is positive (negative) definite if and only if for all $x \in \mathbb{R}^d \setminus \{0\}$ it holds that

$$x^T A x \stackrel{(<)}{>} 0.$$

Theorem (Hessian test)

If a^ is a critical point of f whose second partial derivative exists and are continuous. Then, the*

- 1. If $(Hf)(a^*)$ is definite positive, then a^* is a local minimum of f*
- 2. If $(Hf)(a^*)$ is definite negative, then a^* is a local maximum of f*
- 3. If $(Hf)(a^*)$ is indefinite but $\text{Det}((Hf)(a^*)) \neq 0$, then a^* is a saddle point of f .*
- 4. Otherwise, the test is inconclusive.*

Criteria for verifying Positive Definiteness

1. A is definite Positive if and only if all eigenvalues are positive
2. If the determinant of the principal minors are all positive.

To apply the criteria to check if A is definite negative, just use the the criteria with $-A$.

Exercises: MLE for Normal Random Variables

Let $X_1, \dots, X_n$ be i.i.d. Normal random variables in $\mathbb{R}$ with mean μ and variance σ^2 . Find the maximum likelihood estimator of μ and σ^2 .

Consider pairs of data $(X_i, Y_i)_{i=1}^n$ with $X_i \in \mathbb{R}^d$ and $Y_i \in \mathbb{R}$. Consider the loss function

$$L(\beta_0, \beta) = \sum_{i=1}^n (Y_i - \beta_0 - \beta^\top X_i)^2.$$

Find $\hat{\beta}_0$ and $\hat{\beta}$ that minimises the lost function L .

2. Numerical Optimisation

In most cases we don't have the luxury of having closed formulas

So the idea of finding critical points and verifying if they correspond to a local minimum or maximum is not very systematic

There are other algorithms that are much better at solving different type of optimisation problems. These algorithms usually approximate a solution, so we call them **numerical algorithms**

3. Numerical Algorithms

Suppose we want to optimise $f : U \subset \mathbb{R}^d \rightarrow \mathbb{R}$, in particular, we want to find $\operatorname{argmin}_{\theta \in U} f(\theta)$ or $\operatorname{argmax}_{\theta \in U} f(\theta)$.

The idea is to define $\theta_0 \in U$, and then by using an algorithm, we shall produce a sequence $\theta_1, \theta_2, \dots$ such that

$$\theta_m \rightarrow \operatorname{argmin}_{\theta \in U} f(\theta)$$

as $m \rightarrow \infty$

In practice, several problems arise, and we have to live with them

1. The sequence θ_m may not converge
2. In general we can only show convergence to a local minimiser
3. Sometimes there are not local minima, or it is not well-defined, etc

Most of the algorithms we are interested on can written as

$$\theta_m = \Phi(\theta_{m-1}),$$

i.e. they iterate with the map $\Phi : U \rightarrow U$.

Proposition

Let $\Phi : U \rightarrow U$ be continuous and suppose that $\theta_m \xrightarrow{m \rightarrow \infty} \theta$, then

$$\Phi(\theta) = \theta.$$

This proposition tells us that in this setting we just need to find the so-called **fixed points**.

3. Numerical Algorithms

Gradient Descent

Let's say that we want to minimise $f : U \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$. We assume that U is **open and convex**

A well-known idea is to step on a point $x \in U$. Then, we know that the gradient $(\nabla f)(x)$ tells us in which directions the function f grows. Thus we move in the opposite direction. Hence, we set

$$\theta_{m+1} = \theta_m - \gamma(\nabla f)(\theta_m)$$

where $\gamma > 0$ is the step of the algorithm.

The idea of gradient descent is that we put a ball on top of the function f . The ball has a position x and a height $f(x)$. By gravity the ball has to fall.

Example: Gradient descent for logistic regression

Consider i.i.d. data (X_i, Y_i) with $X_i \in \mathbb{R}^d$ and $Y_i \in \{0, 1\}$.

We model our data by

$$\mathbb{P}(Y = 1|X) = \sigma(\beta^\top X)$$

where $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is given by

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Here we can put a 1 in front of X_i , so we have a β_0 term

The likelihood function of β given the observed data Y_i is given by

$$L(\beta|(X_i, Y_i)_{i=1}^n) = \prod_{i=1}^n \sigma(\beta^\top X_i)^{Y_i} (1 - \sigma(\beta^\top X_i))^{1-Y_i} f_X(X_i)$$

and the log-likelihood is

$$\ell(\beta) = \sum_{i=1}^n Y_i \log [\sigma(\beta^\top X_i)] + \sum_{i=1}^n (1 - Y_i) \log [1 - \sigma(\beta^\top X_i)] + \sum_{i=1}^n \log(f_X(X_i))$$

Since the Gradient Descent algorithm only works for finding minimums, we apply it to $-\ell(\beta)$.

We can verify that

$$\nabla \ell(\beta) = \sum_{i=1}^n (Y_i - \sigma(\beta^\top X_i)) X_i$$

hence the gradient decent gives the following iteration to maximise ℓ (or minimise $-\ell$).

$$\beta_{m+1} = \beta_m + \sum_{i=1}^n (Y_i - \sigma(\beta_m^\top X_i)) X_i$$

Let's implement this algorithm.

We just need to implement the gradient, which can easily be done with the following code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def update_beta(beta, gamma, X, Y):
    a = Y - sigmoid(X @ beta)
    grad = X.T @ a
    return beta + gamma * grad
```

where X represents the matrix with the data X_i as row vectors, with a potential 1 in the first column, to consider intercept.

Detour: some tricks to deal with matrices

When we deal with numerical algorithms in multiple dimensions we will encounter a lot of expressions that can be transformed into matrices expressions. This will make computation faster in languages like R or Matlab, or numerical libraries such as `numpy`.

A few properties to remember $A \in \mathbb{R}^{m \times d}$, $x \in \mathbb{R}^d$

1. $Ax = \sum_{i=1}^d x_i A_i$, where A_i represents the i -column of A
- 2.

$$Ax = \begin{pmatrix} \langle x, A^1 \rangle \\ \langle x, A^2 \rangle \\ \vdots \\ \langle x, A^m \rangle \end{pmatrix}$$

where $\langle \cdot, \cdot \rangle$ represent the usual inner product of vectors in $\mathbb{R}^d$, and A^i represents the i -th row of A

Stopping Criteria

There are several stopping criteria that can be used, but everything is very heuristical

- ▶ **Fixed number of iterations:** simpler way to stop is to fix the number of iterations in advance.
- ▶ **Small progress:** we can stop the algorithm when we are not making more process. Typical rules include

$$\|\theta_{m+1} - \theta_m\|_2 \leq \varepsilon \|\theta_m\|_2 + \delta$$

where $\|\cdot\|_2$ denotes the euclidian norm of θ , and ε and δ are non-negative constant. The first term of the sum evaluates *relative* progress, and the second part *absolute* progress.

- ▶ **Small progress II:** Other way to look progress is to have a look at the function we are minimising, i.e.

$$|f(\theta_m) - f(\theta_{m+1})| \leq \varepsilon |f(\theta_m)| + \delta$$

- ▶ **Small progress III:** stop if $\|\nabla f\|$ is small. Recall that we are looking for critical points (so checking if $\|\nabla f\|$ is small is a good idea.

In practice, we will do a bit of everything. Pictures help, evaluating the algorithm with cross-validation and similar ideas is also good. We will have a lot of trial and error

How to choose γ ?

In theory, we should choose γ as an arbitrary small step as it represents the dx in the derivative.

But if we choose it too small, our algorithm will be very slow. This is something that is not solved and it is still a topic of research.

- **Optimal γ :** Technically, we want to minimise f , so we may try to move to lower values of f as fast as possible. Then, we can choose $\gamma > 0$ such that

$$h(\gamma) = f(\theta_m - \gamma(\nabla f)(\theta_m))$$

is minimised.

Note that $h : (0, \infty) \rightarrow \mathbb{R}$ is a univariate function, so optimising it should not be very hard... in theory. In practice we need to evaluate f and its gradient, and computing the derivative of h can be painfully.

- **Search:** instead of optimising γ , we can just try a few values of γ and look for the best γ . We can try different search schemes. One of them is **linear search**: search in a partition of the interval $(0, \delta)$.

Second Order Approximation

Let $f : U \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$. Assume that U is open and convex and with continuous second order derivatives.

The idea of gradient descent is that

$$f(\theta') \approx f(\theta) + (\theta' - \theta)^\top \nabla f(\theta)$$

and the approximation is very precise when $\theta' \rightarrow \theta$. Then, if we choose $\theta' = \theta - \gamma \nabla f(\theta)$, we get

$$f(\theta - \gamma \nabla f(\theta)) \approx f(\theta) - \gamma \|\nabla f(\theta)\|_2^2 < f(\theta)$$

Of course, we need γ to be small for this approximation to hold but also γ as large as possible to reach the smallest possible value. Note that if the approximation holds then this is the best direction to move

Second Order Approximation

We can use a second order Taylor approximation.

$$f(\theta') \approx f(\theta) + (\theta' - \theta)^\top \nabla f(\theta) + \frac{1}{2}(\theta' - \theta)^\top (Hf)(\theta)(\theta' - \theta).$$

Let $t = \theta' - \theta \in \mathbb{R}^d$, then

$$f(\theta + t) \approx f(\theta) + t^\top \nabla f(\theta) + \frac{1}{2}t^\top (Hf)(\theta)t$$

Let's optimise over $t \in \mathbb{R}^d$ to find the minimum value of $f(\theta) + t^\top \nabla f(\theta) + \frac{1}{2}t^\top (Hf)(\theta)t$

Second Order Approximation

Clearly

$$\nabla_t \left(f(\theta) + t^\top \nabla f(\theta) + \frac{1}{2} t^\top (Hf)(\theta) t \right) = \nabla f(\theta) + (Hf)(\theta) t,$$

so we easily solve $\nabla f(\theta) + (Hf)(\theta) t = 0$ by

$$t = - \left[(Hf)(\theta_m) \right]^{-1} \nabla f(\theta)$$

Setting $\theta = \theta_m$ and $\theta' = \theta_{m+1}$ we get the iteration

$$\theta_{m+1} = \theta_m - \left[(Hf)(\theta_m) \right]^{-1} \nabla f(\theta_m)$$

Some Remarks

1. Computing the Hessian may be painful, but this method uses more information about the function.
2. These methods are called "Newton-type methods", and it is what most libraries use.

Consider data $X_1, \dots, X_n \stackrel{iid}{\sim} f_{\lambda, \beta}$ where

$$f_{\lambda, \beta}(x) = \frac{\beta}{\lambda} \left(\frac{x}{\lambda}\right)^{\beta-1} \exp\left(-\left(\frac{x}{\lambda}\right)^{\beta}\right).$$

Design a GD algorithm for the maximum likelihood estimator of (λ, β) .

4. A bit of theory: fixed points

Let's review a bit of the theory behind GD descent algorithms, and the techniques used to analyse them.

Definition (Fixed Points)

Consider a function $\Phi : U \rightarrow U$, then a point $a \in U$ is a fixed point if and only if

$$\Phi(a) = a.$$

In the simplest GD algorithm we have

$$\theta_{m+1} = \theta_m - \gamma(\nabla f)(\theta_m).$$

Note that if the iteration converges and f has continuous partial derivatives, then θ_m converges to a value θ satisfying

$$\theta = \theta - \gamma(\nabla f)(\theta).$$

That is $(\nabla f)(\theta) = 0$ which is a critical point. We also know that is very unlikely that we are exactly in a value with $(\nabla f)(\theta) = 0$, and thus, GD converges to a local minimum.

We are then interested in the fixed points of $\Phi(\theta) = \theta - \gamma(\nabla f)(\theta)$

We say that a map $\Phi : U \rightarrow U$ with $U \subseteq \mathbb{R}^d$, is contractive if

$$\|\Phi(\theta) - \Phi(\theta')\|_2 \leq \lambda \|\theta - \theta'\|_2$$

with $\lambda < 1$.

Theorem (Banach Fixed Point theorem)

Let $\Phi : U \rightarrow U$ be a contraction, then Φ has a unique fixed point. Moreover, the iteration

$$\theta_{m+1} = \Phi(\theta_m)$$

is such that θ_m converges to a fixed point of Φ when $m \rightarrow \infty$.

Some useful tools: Properties of $\mathbb{R}^d$

A few important properties of sequences of $\mathbb{R}^d$

Cauchy Sequences

A sequence $(a_n)_{n \geq 0}$ in $\mathbb{R}^d$ is called a Cauchy sequence if for every $r > 0$, it exists $N > 0$ such that for any $m, n \geq N$ it holds that

$$\|a_n - a_m\|_2 \leq r.$$

Theorem

$\mathbb{R}^d$ is complete. That is, every Cauchy sequence in $\mathbb{R}^d$ converges to a point in $\mathbb{R}^d$.

For the proof of the Banach Fixed Point Theorem it is also useful to remember that

$$\sum_{n=0}^{\infty} c^n = \frac{1}{1 - c}$$

when $|c| < 1$.

5. EM-algorithms

5. EM-algorithms

Motivation

GD algorithms are good when we can easily compute likelihood or cost functions

But sometimes likelihood involve unknown/unobserved variables, and thus we need to integrate them to incorporate

So the likelihood is very complicated and GD algorithms are not very effective

A complicated likelihood: Easy case

Consider the following model for a data points:

$$X \sim N(\mu_G, 1) \in \mathbb{R}$$

$$G \sim p$$

where p is a discrete distribution on the set $(1, 2, \dots, r)$. The parameters we need to find are $(\mu_1, \dots, \mu_r)$ and $(p_1, \dots, p_r)$. Note that $p_1 + p_2 + \dots + p_r = 1$.

In words, we have r groups, and each group i have their own distribution $N(\mu_i, 1)$

If we observe the pair (X_i, G_i) , then it is easy to write the likelihood of $\mu = (\mu_1, \dots, \mu_r)$ and $p = (p_1, \dots, p_r)$ as

$$L(\mu, p; (X_i, G_i)_{i=1}^n) = \prod_{j=1}^n \frac{p_{G_j}}{\sqrt{2\pi}} \exp\left(-\frac{(X_j - \mu_{G_j})^2}{2}\right)$$

Then we can easily find μ and p by solving the optimisation problem (restricted to $\sum_i p_i = 1$).

A complicated likelihood: Easy case

We want to maximise

$$f(\mu, p) = \sum_{j=1}^n \log(p_{G_j}) - \frac{(X_j - \mu_{G_j})^2}{2} \quad (1)$$

subject to $\sum_{i=1}^r p_i = 1$.

Equivalently, let $\lambda \in \mathbb{R}$, and consider the Lagrange multipliers version of f

$$f(\mu, p, \lambda) = \sum_{j=1}^n \log(p_{G_j}) - \frac{(X_j - \mu_{G_j})^2}{2} + \lambda \left(1 - \sum_{i=1}^r p_i \right)$$

So the critical points of $f(\mu, p, \lambda)$ are now restricted to $\sum_{i=1}^r p_i = 1$ since

$$\frac{d}{d\lambda} f(\mu, p, \lambda) = 1 - \sum_{j=1}^r p_j$$

hence, we just need to analyse the critical points

$$\nabla_{\mu} f(\mu, p, \lambda) = 0$$

$$\nabla_p f(\mu, p, \lambda) = 0$$

$$\frac{d}{d\lambda} f(\mu, p, \lambda) = 0.$$

unsurprisingly, the solution is

$$p_k = \frac{N_k}{n}, \mu_k = \frac{\sum_{j=1}^n X_j \mathbb{1}\{G_j = k\}}{N_k}, \quad k \in \{1, 2, \dots, r\}$$

where N_k is the number of data points that have group k .

5. EM-algorithms

Detour: Gradient Algorithms for optimisation with constraints

Detour: Gradient Algorithms for optimisation with constraints

Consider functions $f : U \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$ restricted to $g : U \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$.

We want to

$$\begin{aligned} &\text{minimise } f(x) \\ &\text{subject to } g(x) = 0 \end{aligned}$$

In typical calculus courses we are taught the method of the Lagrange Multipliers, i.e. we consider

$$f(x, \lambda) = f(x) + \lambda g(x)$$

then we solve the system

$$\nabla_x f(x, \lambda) = 0$$

$$\frac{d}{d\lambda} f(x, \lambda) = 0.$$

Note that $f(x, \lambda)$ has no local minimum, as we can make λ arbitrarily big or small just to make $|f(x, \lambda)|$ as large as desired. Therefore, all the critical points of $f(x, \lambda)$ are no minimal points.

So we cannot apply Gradient Descent

Option 1: Gradient Descent for roots

Let $h : \mathbb{R}^d \rightarrow \mathbb{R}^k$. We want to find x^* such that $h(x^*) = 0$.

Then x^* is a local minimum of $(\sum_{i=1}^k h_i(x))^2$, hence, we can use standard Gradient Descent algorithms to solve this.

In the previous example, we want to solve

$$\nabla_x f(x, \lambda) = 0$$

$$\frac{d}{d\lambda} f(x, \lambda) = 0.$$

so we just solve

$$\min_{x \in U} \left(\sum_{i=1}^d \left(\frac{\partial}{\partial x_i} f(x, \lambda) \right)^2 \right) + \left(\frac{d}{d\lambda} f(x, \lambda) \right)^2$$

so gradient descent is fine!

Option 2: Netwon's method for roots

Let $F : \mathbb{R}^k \rightarrow \mathbb{R}^k$ be a function, and we want to find $x \in \mathbb{R}^k$ such that $F(x) = 0$. Let J_F be the Jacobian matrix given by

$$J_F(x) = \begin{pmatrix} \frac{\partial}{\partial x_1} F_1(x) & \cdots & \frac{\partial}{\partial x_k} F_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} F_k(x) & \cdots & \frac{\partial}{\partial x_k} F_k(x) \end{pmatrix}$$

Then, the iteration

$$x_{m+1} = x_m + (J_F(x_m))^{-1} F(x_m),$$

converges to a zero of F . In our case $F : \mathbb{R}^{d+1} \rightarrow \mathbb{R}^{d+1}$ is given by

$$F(x, \lambda) = \left(\frac{\partial}{\partial x_1} f(x, \lambda), \dots, \frac{\partial}{\partial x_d} f(x, \lambda), \frac{d}{d\lambda} f(x, \lambda) \right)$$

Option 3: Penalisation

To solve

$$\begin{array}{ll} \text{minimise } f(x) \\ \text{subject to } g(x) = 0 \end{array}$$

we can just solve

$$\text{minimise } f(x) + Cg(x)^2$$

and choose C large, so whenever $g(x) \neq 0$, the whole function will be quite large, then, a gradient method would want to stay close to points x with $g(x) \approx 0$.

Option 4: Ascending on λ (Platt & Barr¹)

To solve

$$\begin{aligned} &\text{minimise } f(x) \\ &\text{subject to } g(x) = 0 \end{aligned}$$

we use the following, simultaneous updates to the function $f(x, \lambda) = f(x) + \lambda g(x)$.

$$\begin{aligned} x_{m+1} &= x_m - \nabla_x f(x_m, \lambda_m), \\ \lambda_{m+1} &= \lambda_m + g(x_m) \end{aligned}$$

¹Constrained Differential Optimization, NIPS 1987

5. EM-algorithms

EM-algorithm

A complicated likelihood: The hard case

Let's go back to our example.

We already know that if we observe (X_i, G_i) we have problem that we can solve. Some times we can even have a closed form expression.

The hard case occurs when we do not observe G_i , meaning that we do not know the group of the data point.

Our data is just $(X_i)_{i=1}^n$.

The density function of X_i is then

$$f(x) = \sum_{k=1}^r \frac{p_k}{\sqrt{2\pi}} \exp\left(-\frac{(x - \mu_k)^2}{2}\right)$$

thus, the whole likelihood of μ is given by

$$L(\mu; (X_i)_{i=1}^n) = \prod_{i=1}^n \left(\sum_{k=1}^r \frac{p_k}{\sqrt{2\pi}} \exp\left(-\frac{(X_i - \mu_k)^2}{2}\right) \right)$$

A complicated likelihood: The hard case - Gradient Descent approach

Define

$$\ell(\mu; (X_i)_{i=1}^n) = \sum_{i=1}^n \log \left(\sum_{k=1}^r \frac{p_k}{\sqrt{2\pi}} \exp \left(-\frac{(X_i - \mu_k)^2}{2} \right) \right)$$

For short, we just write $\ell(\mu, p)$. Then we want

$$\begin{aligned} & \max_{\mu, p} \ell(\mu, p) \\ & \text{subject to } \sum_{k=1}^r p_k = 1 \end{aligned}$$

What can we do? GD is not very good as this problem tend to have a lot of local minimum

In the general theory we have data $(X_i, Z_i)_{i=1}^n$ i.i.d from some joint distribution, and we have a set of parameters $\theta \in U$ associated with the distribution that generates the pair (X_i, Z_i) .

For each data points (X_i, Z_i) , we have the so-called *Complete Likelihood*

$$L(\theta; X_i, Z_i) = f_{\theta}(X_i, Z_i)$$

and the *Incomplete Likelihood*

$$L(\theta; X_i) = \begin{cases} \int f_{\theta}(X_i, z) dz & \text{if } Z_i \text{ is continuous} \\ \sum f_{\theta}(X_i, z) & \text{if } Z_i \text{ is discrete} \end{cases}$$

The random variables Z are usually called latent variables.

The EM algorithm is a very efficient algorithm to maximize the incomplete (log)likelihood.

To explain the algorithm, we will use the following shortcut of notation. We write $X = (X_1, \dots, X_n)$ and $Z = (Z_1, \dots, Z_n)$ and $z = (z_1, \dots, z_n)$.

We also write $g_\theta(X, Z) = \prod_{i=1}^n f_\theta(X_i, Z_i)$, etc.

EM-algorithm

1. Set $\theta_0 \in U$
2. Set $k = 0$
3. Find the conditional density $g_{\theta_k}(z|X)$
4. Compute $Q(\theta|\theta_k) = \mathbb{E}_{\theta_k}(\log g_\theta(X, Z)|X, \theta_k)$, where $\mathbb{E}_{\theta_k}$ indicates that $(X, Z) \sim g_{\theta_k}$
5. Find θ that maximises $Q(\theta|\theta_k)$, call it θ_{k+1} .
6. Set $k = k + 1$ and return to Step 3

EM-algorithm for Gaussian Mixtures

Consider a density f in $\mathbb{R}^d$

$$f(x|\phi) = \sum_{k=1}^r p_k f_k(x|\phi_k)$$

where $f_k(x|\phi_k)$ are Gaussian distributions in $\mathbb{R}^d$, but we do not need this for now. In this case the whole set of unknown parameters is $\theta = (p_1, \dots, p_r, \phi_1, \dots, \phi_r)$, where $p_k \geq 0$ satisfies $\sum_{k=1}^r p_k = 1$, and ϕ_k are parameters in some parametric space, probably high-dimensional.

Suppose we observe $X = (X_1, \dots, X_n)$ with the $X_i \stackrel{i.i.d.}{\sim} f$.

The likelihood is, of course,

$$L(\theta; X) = \prod_{i=1}^n \sum_{k=1}^r p_k f_k(x_i; \phi_k)$$

and

$$\ell(\theta; X) = \sum_{i=1}^n \log \left(\sum_{k=1}^r p_k f_k(x_i; \phi_k) \right)$$

However, if we were able to know which group Z_i generated the point X_i , our life would be easier, since the complete likelihood is

$$L(\theta; X, Z) = \prod_{i=1}^n p_{Z_i} f_{Z_i}(X_i; \phi_{Z_i})$$

EM-algorithm for Gaussian mixtures

1. Start with an initial guess $\theta^0 = (p_1^0, \dots, p_r^0, \phi_1^0, \dots, \phi_r^0)$. We use superscript for θ^0 .
2. Set $m = 0$
3. Let $p(k|X_i, \theta^m) = \frac{p_k^m f_k(X_i|\phi_k^m)}{\sum_{z=1}^r p_z^m f_z(X_i|\phi_z^m)}$ the conditional distribution of Z_i given X_i and $\theta = \theta_m$.
4. The $Q(\theta|\theta^m)$ function is given by

$$Q(\theta|\theta^m) = \sum_{i=1}^n \sum_{k=1}^r (\log(p_k) + \log(f_k(X_i|\phi_k))) p(k|X_i, \theta^m)$$

here $\theta = (p_1, \dots, p_r, \phi_1, \dots, \phi_r)$

5. Then maximising $Q(\theta|\theta^m)$ is much easier: $\hat{p}_k = \frac{1}{n} \sum_{i=1}^n p(k|X_i, \theta^m)$, and $\hat{\mu}_k = \frac{\sum_{i=1}^n X_i p(k|X_i, \theta^m)}{\sum_{i=1}^n p(k|X_i, \theta^m)}$ and

$$\hat{\sigma}_k = \frac{\sum_{i=1}^n (X_i - \hat{\mu})(X_i - \hat{\mu})^\top p(k|X_i, \theta^m)}{\sum_{i=1}^n p(k|X_i, \theta^m)}$$

Let's see the Python implementation.

Consider i.i.d. random variables X_i from a distribution f_θ on $[0, \infty)$. The parameter θ can be multivariate. An example can be the Weibull distribution. Suppose that there is another set of i.i.d. random variables C_i also taking values in $[0, \infty)$ that are independent of the X_i 's and they have distribution g . Instead of observing X_i and C_i we observe pairs $(T_i, \Delta_i)_{i=1}^n$, where $T_i = \min(X_i, C_i)$ and $\Delta_i = \mathbb{1}_{\{X_i < C_i\}}$. Write an EM-algorithm to make inference for θ . Test your algorithm when $f_\theta(x) = \frac{1}{\theta} e^{-x/\theta}$ that is, f_θ represent an exponential distribution with mean θ . This data, called right-censored data, occurs in Survival Analysis, where X_i are some times of interest and C_i is a sort of censoring mechanism. E.g. X_i may represent the death time of patients in a clinical trial, and C_i denote a potential time where patients leave the study (e.g. they are too tired and don't want to be in the hospital any longer), therefore, sometimes we just know that by some time C_i the patient was alive but no the actual death time.

Consider the following model for sampling data i.i.d. data:

$$X_i \sim f_\lambda(x) = \lambda e^{-\lambda x}$$

$$Y_i \sim p_{X_i}(k) = \frac{X_i^k e^{-X_i}}{k!}$$

that is, X_i is exponential, and Y_i is a Poisson distribution whose parameter is the random variable X_i . We only observe the variables $(Y_i)_{i=1}^n$ but not the X_i . Write an EM-algorithm to infer λ .

6. The theory of EM-algorithms

Convex and Concave functions

Let's understand a bit of the theory behind EM-algorithms and why it is good.

Definition: Convex Set

A set $U \subseteq \mathbb{R}^d$ is said to be convex if for any two points x and y in U the line segment joining x and y is in U .

Most of the iterative algorithms assume that the set of parameters we are optimising are in a convex set, otherwise there are too many problems.

Definition: Convex and Concave functions

Let $U \subseteq \mathbb{R}^d$ be convex and let $f : U \rightarrow \mathbb{R}$ be a function. We say that

1. f is convex if for every $x, y \in U$ and $t \in [0, 1]$ it holds that
$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y)$$
2. f is concave if for every $x, y \in U$ and $t \in [0, 1]$ it holds that
$$f(tx + (1 - t)y) \geq tf(x) + (1 - t)f(y)$$

Some examples

1. x^2 , x^4 and x^{2k} are convex function in $\mathbb{R}$
2. x^3 , x^5 , and x^{2k+1} are convex in $[0, \infty)$ and concave in $(-\infty, 0]$.
3. $\sqrt{n}$ and $\log(n)$ are concave in $[0, \infty)$ and $(0, \infty)$, respectively.

Some properties

- ▶ Sum of convex (concave) functions is convex (concave)
- ▶ f is convex if and only if $-f$ is concave
- ▶ if $f : (a, b) \rightarrow \mathbb{R}$ and $(d^2/dx^2)f(x) \geq 0$ for all $x \in (a, b)$ then f is convex
- ▶ if $f : (a, b) \rightarrow \mathbb{R}$ and $(d^2/dx^2)f(x) \leq 0$ for all $x \in (a, b)$ then f is concave

Why do we care about convex functions?

— Jensen's inequality —

Let $f : (a, b) \rightarrow \mathbb{R}$ and X be a random variable taking values in (a, b) . Suppose that $\mathbb{E}(|X|) < \infty$ and $\mathbb{E}(|f(X)|) < \infty$. Then

1. if f is convex, then $f(\mathbb{E}(X)) \leq \mathbb{E}(f(X))$
2. if f is concave, then $f(\mathbb{E}(X)) \geq \mathbb{E}(f(X))$.

EM-algorithm, again

We will use some shortcut notation to write the general EM-algorithm.

We denote by $\mathbf{X} = (X_1, \dots, X_n)$ the set of observation and $\mathbf{Z}$ the set of latent variables.

1. In the simplest case for each X_i there is a hidden variable Z_i which can represent a group or something. In this case $\mathbf{Z}$ is a vector with n variables.
2. In Bayesian statistics $\mathbf{Z}$ is a **common** random parameter which is used in the distribution that generates all the X_i .

The options are endless when it is about making complicated things

we will use the letter $\mathcal{P}_\theta$ to represent all the probabilities, so be careful when understanding everything. θ here represents the parameters of the model

1. $\mathcal{P}_\theta(\mathbf{x}, \mathbf{z})$ represent the joint distribution of $(\mathbf{X}, \mathbf{Z})$
2. $\mathcal{P}_\theta(\mathbf{x})$ and $\mathcal{P}_\theta(\mathbf{z})$ represent the marginal distribution
3. $\mathcal{P}_\theta(\mathbf{x}|\mathbf{z})$ and $\mathcal{P}_\theta(\mathbf{z}|\mathbf{x})$ are the conditional distributions
4. Moreover, recall that we will evaluate the function $\mathcal{P}_\theta$ in the random variables $\mathcal{X}$ and $\mathcal{Z}$

In practice we observe $\mathbf{X}$ but no $\mathbf{Z}$. The likelihood is then

$$L(\theta) = \mathcal{P}_{\theta}(\mathbf{X})$$

while the **complete** likelihood, denoted L_c is

$$L_c(\theta) = \mathcal{P}_{\theta}(\mathbf{X}, \mathbf{Z}).$$

The short version of the EM-algorithm is

1. Set $k = 0$ and choose θ^0 .
2. **E-step.** Compute $Q(\theta; \theta^k) = \mathbb{E}_k (\log \mathcal{P}_{\theta}(\mathbf{X}, \mathbf{Z}) | \mathbf{X})$. Here $\mathbb{E}_k$ means that the random variable $\mathbf{Z}$ is sampled from $\mathcal{P}_{\theta^k}$
3. **M-step.** Compute $\operatorname{argmax}_{\theta} Q(\theta; \theta^k)$
4. set $k = k + 1$ and go do 2.

Why does EM-works?

We will apply one of the most clever trick in the whole area of optimisation.

In EM what we we actually wants is a sequence of θ^k such that $\ell(\theta^{k+1}) > \ell(\theta^k)$, i.e. we want to improve the likelihood in every step. If in the current iteration our best parameter is θ^k , we want to

find a value θ to make $\ell(\theta) - \ell(\theta^k)$ as large as possible (of course finding the actual maximum is very hard...)

Why does EM-works? The trick

Note that

$$\begin{aligned}\ell(\theta) &= \log \{ \mathcal{P}_\theta(\mathbf{X}) \} \\ &= \log \left\{ \sum_{\mathbf{z}} \mathcal{P}_\theta(\mathbf{X}|\mathbf{z}) \mathcal{P}_\theta(\mathbf{z}) \right\}\end{aligned}$$

Note the previous sum is the expectation of $\mathcal{P}_\theta(\mathbf{X}|\mathbf{Z})$ conditional on $\mathbf{X}$ and the random variable $\mathbf{Z}$ has distribution $\mathcal{P}_\theta(\mathbf{z})$. We do not like such distribution, because it depends on θ which is something we want to optimise.

The first part of the trick is to do Importance Sampling

$$\begin{aligned}\log \left\{ \sum_{\mathbf{z}} \mathcal{P}_\theta(\mathbf{X}|\mathbf{z}) \mathcal{P}_\theta(\mathbf{z}) \right\} &= \log \left\{ \sum_{\mathbf{z}} \mathcal{P}_\theta(\mathbf{X}|\mathbf{z}) \mathcal{P}_\theta(\mathbf{z}) \frac{\mathcal{P}_{\theta^k}(\mathbf{z}|\mathbf{X})}{\mathcal{P}_{\theta^k}(\mathbf{z}|\mathbf{X})} \right\} \\ &= \log \left\{ \mathbb{E}_{\theta^k} \left(\frac{\mathcal{P}_\theta(\mathbf{X}|\mathbf{Z}) \mathcal{P}_\theta(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})} \middle| \mathbf{X} \right) \right\}\end{aligned}$$

in the previous expectation the only random variable is $\mathbf{Z}$ whose distribution is $\mathcal{P}_{\theta^k}(\mathbf{z}|\mathbf{X})$

Why does EM-works? The trick

Right now we have

$$\ell(\theta) = \log \left\{ \mathbb{E}_{\theta^k} \left(\frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})} \middle| \mathbf{X} \right) \right\}$$

and thus, by using that $\ell(\theta^k) = \log \{ \mathcal{P}_{\theta^k}(\mathbf{X}) \}$ we have

$$\ell(\theta) - \ell(\theta^k) = \log \left\{ \mathbb{E}_{\theta^k} \left(\frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \middle| \mathbf{X} \right) \right\}$$

The second part of the trick is to use Jensen's inequality, since the log is concave, we have

$$\ell(\theta) - \ell(\theta^k) \geq \mathbb{E}_{\theta^k} \left(\log \left\{ \frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \right\} \middle| \mathbf{X} \right)$$

Why does EM-works? The trick

This inequality is not just theory..

It tells us that while we may not optimise $\ell(\theta) - \ell(\theta^k)$ directly, we can improve the lower bound, and that will, indirectly, improve the value $\ell(\theta) - \ell(\theta^k)$.

We can use the same trick to any optimisation problem involving $W(\sum)$ where W is a concave function, especially when $W = \log$.

We want to compute

$$\operatorname{argmax}_{\theta} \mathbb{E}_{\theta^k} \left(\log \left\{ \frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \right\} \middle| \mathbf{X} \right)$$

the log helps a lot here since we can ignore the terms not involving θ , and thus

$$\begin{aligned} \operatorname{argmax}_{\theta} \mathbb{E}_{\theta^k} \left(\log \left\{ \frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \right\} \middle| \mathbf{X} \right) &= \operatorname{argmax}_{\theta} \mathbb{E}_{\theta^k} (\log \{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})\} | \mathbf{X}) \\ &= \operatorname{argmax}_{\theta} \mathbb{E}_{\theta^k} (\log \{\mathcal{P}_{\theta}(\mathbf{X}, \mathbf{Z})\} | \mathbf{X}) \\ &= \operatorname{argmax}_{\theta} Q(\theta; \theta^k) \end{aligned}$$

Why does EM-works? The trick

The final line shows that maximising $Q(\theta; \theta^k)$ is the same as maximising

$$\operatorname{argmax}_{\theta} \mathbb{E}_{\theta^k} \left(\log \left\{ \frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \right\} \middle| \mathbf{X} \right)$$

therefore if we solve the maximisation problem for $Q(\theta; \theta^k)$, we are maximising

$$\mathbb{E}_{\theta^k} \left(\log \left\{ \frac{\mathcal{P}_{\theta}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \right\} \middle| \mathbf{X} \right)$$

also, note that the maximum of the previous quantity is greater or equal to 0, since by choosing $\theta^{k+1} = \theta^k$ the previous quantity is 0.

If we find θ^{k+1} such that

$$\log \left\{ \mathbb{E}_{\theta^k} \left(\frac{\mathcal{P}_{\theta^{k+1}}(\mathbf{X}|\mathbf{Z})\mathcal{P}_{\theta^{k+1}}(\mathbf{Z})}{\mathcal{P}_{\theta^k}(\mathbf{Z}|\mathbf{X})\mathcal{P}_{\theta^k}(\mathbf{X})} \middle| \mathbf{X} \right) \right\} = M > 0$$

then

$$\ell(\theta^{k+1}) \geq \ell(\theta^k) + M$$

showing that EM is always improving our likelihood, and thus reaching a local maximum